Escalabilidade e custo — o balanceamento funcionou? Quanto custa crescer?

Item	Valor
Data	09/11/2026 (testes na AWS em 05 e 06/11/2026)
Ambiente	AWS us-east-1: CloudFront → Application Load Balancer → ECS Fargate Spot (tarefas de 0,25 vCPU e 512 MB, auto scaling de 1 a 3 pela CPU com alvo de 50%) → MongoDB Atlas M0 e S3
Evidências	Pasta relatorios/: escalabilidade-aws-2026-11-06.json, carga-aws-pos-correcao-2026-11-05/, capacidade-consultas-local-2026-11-09.json

1. Resposta curta

Sim, o balanceamento e o auto scaling funcionaram.

- Com a carga, a AWS subiu sozinha de **1 para 3 tarefas**, e o load balancer passou a dividir as requisições entre elas.
- Sem carga, voltou sozinha para **1 tarefa**, cerca de 20 minutos depois.
- Depois de corrigir um defeito de conexão (seção 3), a taxa de erro ficou em **0%**.
- **Ressalva:** na última rodada a carga não ficou igual entre as 3 tarefas. A melhoria proposta está na seção 4.

Custo para crescer (estimativa da seção 5; os preços são referências públicas aproximadas):

Pacientes cadastrados	Custo por mês	Custo por paciente por mês	Custo por 1 milhão de requisições
Demonstração (hoje)	~R$ 170 se ficar ligada 24 h (~R$ 0,23/h)	—	—
100 mil	~R$ 600	R$ 0,006	~R$ 300
1 milhão	**~R$ 2.800** (otimizado: ~R$ 1.450)	**R$ 0,003**	**~R$ 140**
10 milhões	~R$ 13.000	R$ 0,0013	~R$ 65
Brasil inteiro: 1,5 a 2,5 milhões de atendimentos por dia	**~R$ 30.800 a 46.800**	R$ 0,0006 a 0,0007 **por atendimento**	~R$ 73 a 80

Quanto mais gente usa, **mais barato fica por pessoa**: a parte fixa (load balancer, banco) se divide entre mais usuários.

2. Como o balanceamento funciona

Usuários → CloudFront (HTTPS) → Load balancer (divide as requisições) → Tarefas 1, 2 e 3 da API

CloudWatch mede a CPU média: acima de 50%, o auto scaling sobe +1 tarefa; 15 minutos abaixo de 45%, desce 1 (de 1 a 3 tarefas).

- O **load balancer** recebe todas as requisições e distribui entre as tarefas saudáveis. Ele só manda para a tarefa que responde ao /api/health/live.

- O **auto scaling** olha a CPU média a cada minuto. Passou de 50%, sobe mais uma tarefa (até 3). Ficou 15 minutos abaixo de 45%, desce uma.
- Uma tarefa nova leva **cerca de 1,5 minuto** para ficar pronta.

3. Resultados dos testes na AWS

Carga: 20 usuários fazendo login sem parar. O login é a operação mais pesada da API, porque o bcrypt gasta CPU de propósito para proteger as senhas.

Rodada	Quando	Versão da API	C02: 1 tarefa (5 min)	C03: auto scaling (10 min)	Tarefas	Erros
1	05/11, 22:58	Sem a correção	2,8 req/s · p95 9,6 s	6,0 req/s · p95 6,2 s	1 → 3 → 1	**1,65% / 1,09%** (502)
2	05/11, 23:45	**Com a correção**	2,8 req/s · p95 8,9 s	4,7 req/s · p50 1,7 s · p95 9,1 s	1 → 3 → 1	**0% / 0%**
3	06/11, 01:13	Sem a correção (outro deploy)	2,9 req/s · p95 9,9 s	5,9 req/s · p95 6,8 s	1 → 3 → 1	**3,14% / 1,21%** (502)

Linha do tempo do auto scaling (horário de Brasília):

Rodada	Subiu para 2	Subiu para 3	Fim da carga	Desceu para 2	Voltou para 1
1	23:04	23:10	23:13	23:31	23:34
2	23:51	23:57	00:00	00:21	00:24

O que os números mostram

1. **Escala de verdade.** Na rodada 1, com 3 tarefas no fim da carga, a vazão chegou a **8,4 logins/s**. É 3 vezes a de 1 tarefa (2,8/s), ou seja, crescimento linear.
2. **Defeito encontrado e corrigido.** Os erros 502 vinham do próprio load balancer; a API não registrou nenhum erro.
   - **Causa:** o Node fechava conexões paradas em 5 s, e o load balancer as reaproveitava por até 60 s.
   - **Correção:** a API agora segura a conexão por 65 s.
   - **Prova:** a rodada 2, com a correção, teve **0 erros em 3.659 requisições**. A rodada 3 usou um deploy sem a correção e os erros voltaram, o que confirma a causa.
3. **Monitoramento funcionou.** O alarme de lentidão (p95 acima de 2 s) mandou e-mail durante a carga e avisou de novo quando normalizou.

4. Ressalva: carga desigual entre as tarefas

Na rodada 2, com 3 tarefas:

- o tempo **típico** caiu de 7,3 s para **cerca de 1 s** (p50);
- mas os **5% mais lentos** continuaram perto de 8,5 s (p95);
- a CPU média ficou em 87%, e não em 100%.

Leitura: uma das tarefas recebeu mais trabalho que as outras e acumulou fila. O load balancer divide por número de requisições (*round robin*), mas um login demora muito mais que as outras requisições.

Melhoria proposta (não aplicada): trocar o algoritmo do load balancer para `least_outstanding_requests`, que manda cada requisição para a tarefa com menos trabalho pendente. É uma linha no `infra/lib/api-stack.ts`, e depois é só repetir o C03.

5. Quanto custa se chegar mais gente

5.1 Capacidade de uma tarefa (medida)

Operação	1 processo local (Ryzen 5 5600GT)	1 tarefa na AWS (0,25 vCPU)
Login (bcrypt)	16,9 req/s	**2,8 req/s** (medido)
Abrir prontuário (com auditoria)	599 req/s	~100 req/s (estimado)
Agenda do médico	1.116 req/s	~185 req/s (estimado)
Carteira digital (app)	854 req/s	~140 req/s (estimado)
Atendimentos do paciente (app)	1.511 req/s	~250 req/s (estimado)

A coluna da AWS aplica a mesma proporção medida no login: 2,8 ÷ 16,9 = 0,17.

As consultas são **35 a 90 vezes mais leves que o login**. Para a estimativa:

- usei uma mistura de **5% logins e 95% consultas**, considerando todas as consultas tão pesadas quanto abrir o prontuário (cenário pessimista);
- isso dá cerca de **36 req/s por tarefa** com a CPU em 100%;
- com o alvo do auto scaling em 50%, sobram **~18 req/s por tarefa**.

5.2 Premissas de uso

Premissa	Valor	Por quê
Requisições por paciente por mês	20	Abrir o app 2 vezes por mês (~6 requisições cada) mais as consultas do médico no atendimento
Horário de uso	12 h por dia	Unidades de saúde e uso do app durante o dia
Pico	3 vezes a média	Manhãs de segunda-feira, campanhas de vacinação
Documentos	2 por paciente por ano, 300 KB cada	Receitas e laudos em PDF
Dólar	R$ 5,50	—

Pico de requisições por segundo = pacientes × 20 ÷ (30 dias × 12 h × 3.600 s) × 3.

Pacientes	Requisições por mês	Pico	Tarefas necessárias
100 mil	2 milhões	4,6 req/s	2 (mínimo para alta disponibilidade)
1 milhão	20 milhões	46 req/s	2 a 5 (média de 3)
10 milhões	200 milhões	463 req/s	4 a 10 tarefas maiores (1 vCPU), média de 7

5.3 Custo mensal estimado (US$)

Os preços são referências públicas da AWS (us-east-1, sob demanda) e do MongoDB Atlas, aproximados em nov/2026: Fargate US$ 0,0123/h por tarefa de 0,25 vCPU, load balancer US$ 0,0225/h, IP público US$ 0,005/h, CloudFront América do Sul US$ 0,022 por 10 mil requisições acima do nível gratuito, S3 US$ 0,023/GB. **Antes de usar em proposta oficial, confira na AWS Pricing Calculator e na página de preços do Atlas.**

Item	100 mil	1 milhão	10 milhões
ECS Fargate, sob demanda (sem Spot, para não cair)	18	27	252
Load balancer	17	18	24
IPs públicos	15	18	33
CloudFront (10 milhões de requisições/mês grátis)	0	22	418
MongoDB Atlas	58 (M10)	394 (M30)	~1.460 (M50)
S3 (documentos)	2	16	160
CloudWatch (logs, alarmes)	1	3	15
Redis (limite de login entre tarefas)	—	12	23
Total por mês	**~US$ 110 (R$ 600)**	**~US$ 510 (R$ 2.800)**	**~US$ 2.390 (R$ 13.100)**
Por paciente por mês	R$ 0,006	R$ 0,003	R$ 0,0013
Por 1 milhão de requisições	**R$ 300**	**R$ 140**	**R$ 65**

Para comparar, a demonstração de hoje (1 tarefa Spot e Atlas M0 gratuito) custa cerca de **US$ 0,042/h**: R$ 0,23 por hora, ou ~R$ 170 por mês se ficar ligada direto. Por isso ela só sobe para testes e apresentações.

5.4 Onde está o custo e como baixar (cenário de 1 milhão)

- **O banco é ~77% do custo.** O que mais ocupa espaço é a auditoria (cada acesso ao prontuário vira um registro).
  - Guardar os últimos 6 meses no Atlas e arquivar o resto no S3, que é ~10 vezes mais barato, permite usar o **M20 (~US$ 146)** em vez do M30.
  - O total cai para **~US$ 260/mês (~R$ 1.450)**.
- **Fargate Spot** em parte das tarefas (mantendo 2 sob demanda) economiza mais ~US$ 10.
- **CloudFront:** acima de dezenas de milhões de requisições, vale avaliar os planos de preço fixo do CloudFront.
- **Região São Paulo** (sa-east-1), recomendada pela LGPD em produção: computação e load balancer custam cerca de 30% a 50% a mais que na Virgínia.

5.5 O que muda na arquitetura para 1 milhão

Hoje (demonstração)	Para 1 milhão	Motivo
Atlas M0 gratuito	M30 (ou M20 com arquivamento)	O M0 tem 512 MB e limite de ~100

Hoje (demonstração)	Para 1 milhão	Motivo
		operações/s
Fargate Spot, 1 a 3 tarefas	Sob demanda, 2 a 5 tarefas, em 2 zonas	Não pode cair numa interrupção do Spot
Limite de login na memória de cada tarefa	Redis compartilhado (ElastiCache)	Com 5 tarefas, o limite por tarefa vira 5 vezes maior
Round robin	`least_outstanding_requests`	Seção 4
Auditoria sem prazo	6 meses no banco, depois arquivo no S3	Seção 5.4

O código da API não precisa mudar para crescer: só a configuração da infraestrutura em `infra/`, que já é código (AWS CDK).

6. Cenário Brasil: 1,5 a 2,5 milhões de atendimentos por dia

Premissa do grupo: contando quem vai ao pronto-socorro, faz exames ou fica internado, o movimento total nos hospitais brasileiros (rede pública e particular) fica em torno de **1,5 a 2,5 milhões de pessoas por dia**. É uma estimativa trazida pelo grupo, não um dado oficial conferido. Aqui ela vira o pior caso: **todos os hospitais do país usando a plataforma**.

6.1 Quantas requisições cada atendimento gera

O que acontece no atendimento	Requisições
Médico abre o prontuário	1
Médico consulta o histórico do paciente	1
Agenda do médico (atualizada entre um paciente e outro)	1
Registrar o atendimento (com alergias, remédios, condições)	1
Anexar receita ou laudo	1
Baixar ou conferir um documento (farmácia, outro hospital)	1
Paciente consulta o app depois (carteira e atendimentos)	2
Consultas e gravações	**8**
Logins: médico (1 a cada 20 pacientes, 0,05) + metade dos pacientes entrando no app (0,5)	0,55
Total por atendimento	**8,55**

Os logins são só 6% das requisições, mas pesam mais da metade da CPU, por causa do bcrypt.

6.2 Volume e servidores necessários

Premissas: 70% do movimento entre 7h e 19h, e pico (segunda de manhã) com o dobro da média desse horário. Capacidade medida: ~62 req/s por vCPU com o auto scaling em 50% (seção 5.1).

	1,5 milhão de atendimentos por dia	2,5 milhões de atendimentos por dia
Requisições por dia	12,8 milhões	21,4 milhões
Requisições por mês	385 milhões	641 milhões
Média entre 7h e 19h	208 req/s	346 req/s
Pico	**416 req/s**	**693 req/s**
vCPUs no pico	6,7 → **7 tarefas de 1 vCPU**	11,2 → **12 tarefas de 1 vCPU**
vCPUs de madrugada (mínimo para não cair)	2	2 a 3
Tráfego de saída por mês (respostas de ~5 KB)	~1,9 TB	~3,2 TB
Documentos novos por dia (metade dos atendimentos, 300 KB cada)	750 mil (~6,8 TB/mês)	1,25 milhão (~11 TB/mês)
Crescimento do banco (atendimento + auditoria, ~2,3 KB por atendimento)	~3,4 GB/dia (~1,2 TB/ano)	~5,6 GB/dia (~2 TB/ano)

O que muda na infraestrutura: basta subir o máximo do auto scaling de 3 para ~15 tarefas, com tarefas de 1 vCPU, no infra/lib/api-stack.ts. O código da API não muda.

O que muda de verdade é o **banco**. Nessa escala, o MongoDB precisa de um cluster grande ou dividido em partes (*sharding*). O histórico antigo vai para armazenamento mais barato: Online Archive do Atlas, ou S3 Glacier para os documentos.

6.3 Custo mensal estimado (US$)

Mesmas referências de preço da seção 5.3 (aproximadas). Os documentos estão calculados no fim do 1º ano. Os arquivos com mais de 90 dias vão para o S3 Glacier Instant Retrieval (US$ 0,004/GB).

Item	1,5 milhão/dia	2,5 milhões/dia	Observação
ECS Fargate (API)	104	157	~70 a 106 vCPU-hora por dia
Load balancer	32	42	Inclui as unidades de capacidade (LCU)
IPs públicos	22	29	
CloudFront (requisições + tráfego)	926	1.632	US$ 0,022 a cada 10 mil requisições na América do Sul
MongoDB Atlas	**~3.200**	**~4.500**	M60 ou cluster dividido, com backup. **É a estimativa mais incerta**
S3 (documentos)	**~1.000**	**~1.670**	Armazenamento + downloads + envios. Cresce a cada mês
CloudWatch	25	35	
Redis (limite de login entre tarefas)	47	47	2 nós pequenos
WAF (firewall contra ataques)	240	395	Recomendado em escala nacional

Item	1,5 milhão/dia	2,5 milhões/dia	Observação
Total por mês	**~US$ 5.600 (R$ 30.800)**	**~US$ 8.500 (R$ 46.800)**	
Otimizado: API direto no load balancer, com domínio próprio, sem CloudFront	~US$ 4.840 (R$ 26.600)	~US$ 7.160 (R$ 39.400)	Sem a taxa por requisição do CloudFront
Por ano	**~R$ 370 mil**	**~R$ 560 mil**	

6.4 Quanto custa cada atendimento

	1,5 milhão/dia	2,5 milhões/dia
Atendimentos por mês	45 milhões	75 milhões
Custo por atendimento	**R$ 0,0007** (menos de 1 décimo de centavo)	**R$ 0,0006**
Custo por 1 milhão de requisições	~R$ 80	~R$ 73
Custo por brasileiro por mês (~203 milhões de habitantes)	R$ 0,00015	R$ 0,00023

6.5 Leitura

- **A API não é o problema.** Os servidores da API são só ~2% do custo: 7 a 12 tarefas de 1 vCPU no pico dão conta do país inteiro. O auto scaling medido na seção 3 é o mesmo mecanismo, só com o máximo maior.
- **O custo está nos dados.** O banco é ~53% a 57% do total, e os documentos, cerca de 20%. A conta cresce todo mês, porque prontuário e auditoria não podem ser apagados. A LGPD e as regras do CFM pedem a guarda do prontuário por muitos anos. A política de arquivamento (dados antigos em armazenamento barato) é o que mantém o custo sob controle.
- **Ordem de grandeza:** mesmo no pior caso, com todos os hospitais do Brasil, fica em torno de **R$ 30 a 47 mil por mês**, menos de 1 décimo de centavo por atendimento.
- **Limites desta estimativa:**
  - as premissas de uso (requisições por atendimento, horário de pico, tamanho dos arquivos) são do grupo, não medidas em hospital real;
  - a capacidade por vCPU foi extrapolada da medição local;
  - o preço do Atlas em escala depende de negociação;
  - não entram a equipe, o suporte e a integração com os sistemas dos hospitais.

7. Conclusão

- O **auto scaling e o load balancer funcionam**:
  - a API sobe e desce sozinha conforme a carga;
  - a vazão cresce na proporção do número de tarefas;
  - depois da correção do keep-alive, os erros foram zero.

- **O ponto de melhoria** é a distribuição desigual da carga entre as tarefas. Ela tem solução simples (seção 4) e ainda precisa ser testada.
- **Para 1 milhão de pacientes, a estimativa é de ~R$ 2.800 por mês** (~R$ 1.450 com as otimizações): menos de **R$ 0,003 por paciente por mês**, ou **~R$ 140 por milhão de requisições**. O maior custo é o banco de dados, não os servidores da API.
- **No cenário Brasil inteiro (1,5 a 2,5 milhões de atendimentos por dia), a estimativa é de R$ 30 a 47 mil por mês**, menos de R$ 0,001 por atendimento (seção 6).